

Pipeline MIPS 處理器之效能優化研究： 涵蓋 13 項貢獻之綜合分析

JUNYI LIU

Abstract—本論文針對 5-stage pipeline MIPS 處理器進行全面性效能優化研究。我們實作並評估 13 項貢獻，包含 Data Forwarding、Branch Prediction、SIMD 平行處理、CORDIC 三角函數運算、Polynomial exp 逼近、L1 Cache，以及用於微積分運算的 ODE Solver。評估採用兩種方法：(1) cycle-accurate RTL simulation，使用 MiBench bitcount benchmark；(2) analytical model-based projection，使用 SPEC CPU2006 instruction mix ratios。結果顯示，在 MiBench 上實測達到 $2.61\times$ 加速，而在 memory-intensive SPEC workloads 上預估可達 $34.84\times$ 加速。Design of Experiments (DOE) 分析揭示 Cache 提供最大的單項改進 ($2.15\times$)，且 Cache+SIMD 展現正向交互作用 (synergy factor 1.20 under our cost model)。

Index Terms—MIPS, Pipeline, Forwarding, Branch Prediction, Cache, SIMD, CORDIC, ODE Solver, MiBench

I. 緒論

現代處理器設計仰賴多種優化技術來彌補 CPU 與記憶體之間的效能差距——即所謂的「Memory Wall」問題 [1]。本專案為 5-stage MIPS pipeline 實作 13 項優化。本節為不熟悉計算機架構概念的讀者提供基礎背景知識。

A. 什麼是 MIPS？

MIPS (Microprocessor without Interlocked Pipeline Stages) 是一種 32-bit RISC (Reduced Instruction Set Computer) 架構，由 John Hennessy 於 1981 年在 Stanford University 開發 [4]。MIPS 在計算機架構教育中被廣泛使用，原因如下：

- **簡潔設計**：固定 32-bit 指令格式，僅有三種指令類型 (R, I, J)
- **教育標準**：被 Patterson & Hennessy 教科書採用，為計算機架構之「聖經」
- **實務應用**：廣泛應用於嵌入式系統、路由器 (Cisco)、遊戲主機 (PlayStation 1/2) 及 IoT 裝置

MIPS 基本指令類型：

- **R-type** (Register)：add \$t0, \$t1, \$t2 — 暫存器之間的算術運算
- **I-type** (Immediate)：lw \$t0, 0(\$t1) — 記憶體存取及立即值
- **J-type** (Jump)：j label — 無條件跳躍

B. 5-Stage Pipeline 架構

Pipelining 是一種允許多條指令重疊執行的技術，類似於工廠的組裝線。不同於完成一條指令後才開始下一條，每個 stage 同時處理不同的指令。



Fig. 1. 5-Stage MIPS Pipeline 架構。Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) 儲存 stages 之間的中間結果，實現指令重疊。

五個 Stage 說明：

- 1) **IF (Instruction Fetch)**：使用 Program Counter (PC) 從記憶體讀取指令。PC $\leftarrow$ Memory $\rightarrow$ Instruction Register
- 2) **ID (Instruction Decode)**：解碼指令位元並從 register file 讀取運算元。也執行 branch 位址計算
- 3) **EX (Execute)**：執行 ALU 運算 (算術、邏輯、load/store 之位址計算)
- 4) **MEM (Memory Access)**：為 load (lw) 和 store (sw) 指令進行資料記憶體存取。其他指令直接通過
- 5) **WB (Write Back)**：將結果寫回 register file

Pipeline 時序分析：在理想的 5-stage pipeline 中，經過初始填充時間 (5 個 cycle) 後，每個 cycle 完成一條指令。這給出理論 CPI (Cycles Per Instruction) 為 1.0。然而，hazards 使得實際上無法達到此理想值。

C. 三種 Pipeline Hazards

Hazards 是阻止下一條指令在下一個 clock cycle 執行的情況。理解 hazards 是處理器優化的基礎。

1) **Data Hazards (RAW, WAR, WAW)**: Data hazards 發生在指令依賴於先前尚未完成之指令的結果時。

RAW (Read-After-Write) — 最常見的 hazard：

```
1 add $t1, $t2, $t3    # Writes $t1 in WB (cycle 5)
2 sub $t4, $t1, $t5    # Reads $t1 in ID (cycle 3) - HAZARD!
```

sub 指令需要 \$t1 的值，但 add 尚未寫入。無 forwarding，pipeline 必須 stall 2 個 cycle。有 forwarding，結果可直接從 EX/MEM register 旁路傳送。

WAR (Write-After-Read) 和 **WAW (Write-After-Write)**：這些主要是 out-of-order execution 處理器的問題，不是我們這種簡單 in-order pipeline 的問題。

2) **Control Hazards (Branch Hazards)**: Control hazards 發生在處理器因 branch 決定尚未做出而無法確定下一條要 fetch 的指令時。

範例：

```
1 beq $t0, $t1, target # Branch decision in EX stage
2 add $t2, $t3, $t4    # Fetched speculatively - wrong?
3 sub $t5, $t6, $t7    # Also speculatively fetched
```

若 branch taken，則兩條已 fetch 的指令必須被 flush (丟棄)，浪費 cycles。**Branch prediction** 預測結果以減少此 penalty。

TABLE I
13 項優化貢獻分類

類別	貢獻
Pipeline Hazard Mitigation	#1 Performance Testbench, #4 Quantitative Analysis, #6 Branch Prediction, #7 Comprehensive DOE
Data Hazard Resolution	#3 Data Forwarding
平行處理與加速	#2 Hardware Unrolling, #5 SIMD ALU Expansion, #9 CORDIC, #10 Polynomial Exp
記憶體階層	#11 L1 Cache, #12 Integrated Analysis
解析器增強	#8 Parentheses Support
微積分運算	#13 ODE Solver (Euler Method)

3) *Structural Hazards*: Structural hazards 發生在硬體資源不足以支援所有 active 指令時。我們的設計使用分離的 instruction 和 data memories (Harvard architecture) 來避免此 hazard。

D. Memory Wall 問題

「Memory Wall」指的是處理器速度與記憶體速度之間日益增長的差距 [1]：

- **CPU 效能**：每年改進約 50%（直到 2003 年）
- **DRAM Latency**：每年僅改進約 7%
- ，單次記憶體存取可能耗費 100–300 個 CPU cycles

解決方案：Cache memory hierarchy。透過將經常存取的資料保存在小型、快速的 caches (L1, L2, L3) 中，處理器大部分時間可以避免緩慢的 main memory 存取。我們的 L1 cache 實作將平均記憶體存取時間降低 26×（基於 97.22% hit rate）。

E. 問題陳述

本專案實作的 baseline MIPS pipeline 存在以下問題：

- **Data Hazards**：RAW dependencies 在無 forwarding 時需要 2-cycle stalls，將 CPI 從 1.0 增加到 1.82
- **Control Hazards**：每次 mispredicted branch 造成 pipeline flush (1–2 cycle penalty)，顯著影響 loops
- **Memory Latency**：無 cache 時，每次 load/store 需要 100 cycles，主導執行時間
- **有限的平行性**：Scalar ALU 每個 cycle 僅處理一個運算，未充分利用硬體

F. 貢獻總覽

本專案實作 13 項獨立優化，彙整於表 I。每項優化針對特定瓶頸，其綜合效果使用 Design of Experiments (DOE) 方法分析。

II. 研究方法

本節說明用於量測和預估處理器效能的評估方法。我們首先介紹基本的效能指標 (CPI)，然後描述使用的 benchmarks，最後說明我們的兩種評估方法：直接量測和分析預估。

TABLE II
評估使用的 BENCHMARKS

類型	名稱	用途
標準演算法	Bubble Sort [7]	Contrib 1, 4: Baseline CPI
嵌入式	MiBench bitcount [10]	Contrib 12: 實際執行
Workload Mix	SPEC CPU2006 [2]	Contrib 7, 12: 預估

A. 理解 CPI (Cycles Per Instruction)

CPI 是評估處理器效能的主要指標。它量測平均執行一條指令所需的 clock cycles 數。

基本 CPI 公式：

$$CPI = \frac{\text{Total Cycles}}{\text{Total Instructions}} \quad (1)$$

理想 vs 實際 CPI：

- **理想 CPI**：在完美的 5-stage pipeline 中無 hazards，CPI = 1.0（pipeline 填滿後每 cycle 完成一條指令）
- **實際 CPI**：由於 stalls 和 flushes，實際 CPI > 1.0。我們的 baseline 量測 CPI = 1.82

CPI 分解公式：CPI 可以分解為各貢獻因素：

$$CPI = CPI_{\text{ideal}} + CPI_{\text{stall}} = 1.0 + \frac{\text{Stall Cycles}}{\text{Instructions}} \quad (2)$$

相關指標：

- **IPC (Instructions Per Cycle)**：IPC = $\frac{1}{CPI}$ — 越高越好
- **Speedup**：Speedup = $\frac{CPI_{\text{baseline}}}{CPI_{\text{optimized}}}$
- **Pipeline Efficiency**：Efficiency = $\frac{\text{Ideal CPI}}{\text{Actual CPI}} = \frac{1}{CPI}$

計算範例：我們的 baseline Bubble Sort 測試：

$$\begin{aligned} \text{Total Cycles} &= 255 \\ \text{Total Instructions} &= 140 \\ CPI &= \frac{255}{140} = 1.82 \\ IPC &= \frac{1}{1.82} = 0.549 \\ \text{Pipeline Efficiency} &= \frac{1}{1.82} = 55\% \end{aligned}$$

這意味著 45% 的 cycles 因 stalls 而「浪費」。

B. Benchmark 選擇標準

好的 benchmark 應該 [1]：

- 1) **代表性**：反映真實世界 workload 特徵
- 2) **可重現性**：結果可被他人驗證
- 3) **可攜性**：可在不同系統上執行
- 4) **可量測性**：執行可精確計時

我們採用三種 benchmark 類別，各有不同目的：

1) *Bubble Sort*: Bubble Sort 是經典的 $O(n^2)$ 排序演算法，重複走訪列表、比較相鄰元素並在需要時交換 [7]。我們使用它因為：

- 包含迴圈、條件判斷和記憶體存取 — 測試所有 pipeline stages
- 容易用組合語言實作
- 因相依運算產生許多 RAW hazards

2) *MiBench (Michigan Benchmark Suite)*: MiBench 是 University of Michigan 於 2001 年發布的免費開源嵌入式系統 benchmark suite [10]。我們使用 bitcount 演算法 (Brian Kernighan's Bit Counting)：

```
1 // Brian Kernighan's Bit Counting Algorithm
2 int bitcount(unsigned int n) {
3     int count = 0;
4     while (n) {
5         count++;
6         n = n & (n - 1); // Clear lowest set bit
7     }
8     return count;
9 }
```

關鍵優勢：該演算法夠簡單，可以實作為 Verilog testbench 並實際執行於我們的處理器 RTL，提供真實的 cycle counts。

3) *SPEC CPU2006*: SPEC CPU2006 是 Standard Performance Evaluation Corporation 的業界標準處理器 benchmark suite。由於執行完整 SPEC 需要：

- OS 支援 (Linux/Windows)
- 商業授權
- 完整系統模擬

我們改用 Phansalkar et al. [2] 發表的 **instruction mix ratios** 進行分析預估，這是有效的學術方法。

C. Mix-Based 效能預估

定義：一種使用指令類型比例結合每指令執行成本來預估總 cycles 的效能預測方法。

公式：

$$\text{Cycles}_{\text{est}} = \sum_{i \in \{\text{types}\}} \text{Count}_i \times \text{CostPerInstr}_i \quad (3)$$

逐步範例 (10,000 instructions, SPEC Overall mix)：

- 1) **Instruction counts**：Branch: $10000 \times 0.15 = 1500$ ，Memory: $10000 \times 0.496 = 4960$ ，Compute: $10000 \times 0.354 = 3540$
- 2) **無優化** (baseline costs)：Branch: 3 cycles，Memory: 100 cycles，Compute: 1 cycle
- 3) **Baseline cycles**： $1500 \times 3 + 4960 \times 100 + 3540 \times 1 = 4500 + 496000 + 3540 = 504040$
- 4) **有優化**：Memory cost 降到 ~ 1.2 cycles (cache)，Branch cost 降到 1.65 (prediction)
- 5) **Optimized cycles**： $1500 \times 1.65 + 4960 \times 1.2 + 3540 \times 0.125 = 2475 + 5952 + 443 = 8870$
- 6) **Speedup**： $504040/8870 = 56.8\times$ (理論最大值)

為何 MiBench (2.61 $\times$) 與 SPEC 預估 (29.57 $\times$) 差異這麼大？

- MiBench bitcount 是簡單的 bit-manipulation 程式，幾乎沒有記憶體存取 — 主要受益於 Forwarding

TABLE III
實際執行 vs Mix-Based 預估比較

面向	實際執行	Mix-Based 預估
方法	執行完整程式	比例乘以成本
資料來源	自行量測的 cycles	已發表論文
準確度	高 (ground truth)	預估 (可能高估)
優勢	可靠、可驗證	無需完整系統
本專案	MiBench: 2.61 $\times$	SPEC: 29.57 $\times$

TABLE IV
SPEC CPU2006 INSTRUCTION MIX RATIOS (來自已發表研究)

Workload	Branch	Memory	Compute
SPEC FP Average	8%	45%	47%
SPEC INT mcf	18%	62%	20%
SPEC INT gobmk	22%	48%	30%
SPEC Overall	15%	49.6%	35.4%

- SPEC 預估假設 49.6% memory instructions — Cache 對 memory-bound workloads 提供巨大優勢
- 這證明優化效果高度依賴 **workload 特徵**

D. SPEC CPU2006 Instruction Mix

我們使用 Phansalkar et al. [2] 和 Limaye & Adegbiya [6] 的 instruction mix ratios，代表不同 workload 類型：

(memory-intensive) 和 gobmk (branch-heavy) 代表極端情況；SPEC Overall 是加權平均。

E. 實驗設置

- **模擬工具**：AMD Xilinx Vivado 2025.2，Behavioral Simulation mode
- **HDL 語言**：Verilog-2005
- **架構**：5-stage pipeline (IF, ID, EX, MEM, WB)
- **資料寬度**：32-bit instructions and data
- **Clock**：Functional simulation (cycle-accurate，無 timing delays)
- **Memory Model**：分離的 instruction 和 data memories (Harvard architecture)

F. Testbench 總覽

表 V 顯示每項貢獻的 testbench。

III. 貢獻細節

本節詳細描述 13 項貢獻的實作細節與驗證結果。

A. Contribution 1 & 4: Performance Testbench 與 Quantitative Analysis

testbench 基礎設施，用於量測 pipeline 效能指標 (CPI, IPC, stall cycles) 並量化 data forwarding 的影響。

TABLE V
各貢獻的 TESTBENCH 檔案

#	Testbench	驗證項目
1	tb_performance.v	CPI/IPC, Bubble Sort
2	tb_simd_add.v	8-lane parallel add
3	tb_simd_alu.v	40 SIMD operations
4	tb_performance.v	Stall breakdown analysis
5	tb_simd_alu_expanded.v	5 ops + Expression
6	tb_branch_prediction.v	4 branch patterns
7	tb_comprehensive.v	16 DOE configs
8	tb_parentheses.v	Shunting-yard
9	tb_cordic.v	5 canonical angles
10	tb_poly_exp.v	4 exp() test cases
11	tb_cache_hierarchy.v	L1/L2/L3 AMAT
12	tb_mibench_bitcount.v	Actual execution

1) 為何需要 *Performance Testbench*: 沒有儀器化, 我們無法量測實際效能。Testbench 添加:

- **Cycle Counter**: 每個 clock cycle 遞增
- **Instruction Counter**: 當指令完成 (達到 WB stage) 時遞增
- **Stall Counter**: 追蹤 pipeline stalled 的 cycles

Testbench 架構 (逐行說明):

```

1 module tb_performance;
2   reg clk, rst;           // Clock and reset signals
3   wire [31:0] cycle_count; // Total cycles elapsed
4   wire [31:0] instr_count; // Instructions completed
5
6   // Instantiate Device Under Test (DUT)
7   topLevelCircuit DUT (
8     .clk(clk),
9     .rst(rst)
10  );
11
12  // Clock generation: 10ns period (100MHz)
13  always #5 clk = ~clk;
14
15  initial begin
16    clk = 0;
17    rst = 1; // Assert reset
18    #20;    // Hold reset for 20ns
19    rst = 0; // Release reset
20    wait(done); // Wait for program completion
21    // Display results
22    $display("Cycles: %d, Instructions: %d",
23            cycle_count, instr_count);
24    $display("CPI = %f",
25            real'(cycle_count) / real'(instr_count));
26    $finish;
27  end
28 endmodule

```

2) *Data Forwarding*: 解決 *RAW Hazards*: Data forwarding (也稱 *bypassing*) 透過將結果從較晚的 pipeline stages 直接路由回較早的 stages, 消除大多數 data hazard stalls。

RAW Hazard 範例 (無 Forwarding):

```

1 # Cycle:      1    2    3    4    5    6    7
2 add $t1,$t2,$t3 IF ID EX MEM WB
3 sub $t4,$t1,$t5 IF ID *** EX MEM WB
4 #

```

無 forwarding, sub 指令必須等到 add 在 cycle 5 寫入 \$t1 到 register file。這造成 2-cycle stall (顯示為 ***)。

RAW Hazard 範例 (有 Forwarding):

```

1 # Cycle:      1    2    3    4    5    6
2 add $t1,$t2,$t3 IF ID EX MEM WB
3 sub $t4,$t1,$t5 IF ID EX MEM WB
4 #

```

有 forwarding, add 的結果在 EX 結束時即可用 (儲存在 EX/MEM register)。此值直接 forward 到 sub 的 EX stage 輸入, 消除 stall。

TABLE VI
DATA FORWARDING 的 CPI 降低

Configuration	Cycles	Instr	CPI	IPC
No Forwarding	255	140	1.82	0.549
With Forwarding	182	144	1.26	0.791

3) *Forwarding Unit* 邏輯: Forwarding Unit 決定何時及從何處 forward 資料:

Forwarding 條件 (Verilog):

```

1 // Forward from EX/MEM stage (1-cycle-old result)
2 // Priority: EX/MEM has most recent value
3 assign ForwardA_EXMEM =
4   (EX_MEM_RegWrite) && // MEM will write
5   (EX_MEM_Rd != 0) && // Not $zero
6   (EX_MEM_Rd == ID_EX_Rs); // Match source
7
8 // Forward from MEM/WB stage (2-cycle-old result)
9 // Only if EX/MEM didn't already forward
10 assign ForwardA_MEMWB =
11   (MEM_WB_RegWrite) && // WB will write
12   (MEM_WB_Rd != 0) && // Not $zero
13   !(ForwardA_EXMEM) && // EX/MEM didn't forward
14   (MEM_WB_Rd == ID_EX_Rs); // Match source

```

為何檢查 $Rd \neq 0$? 在 MIPS 中, register \$zero (register 0) 硬連線為零且不能被寫入。從寫入 \$zero 的操作 forward 會是錯誤的。

4) *Load-Use Hazard*: *Forwarding* 無法解決的情況: 即使有 forwarding, 有一種情況仍需要 stall: 當 lw (load word) 指令緊接著一條使用載入值的指令時。

為何? 載入的資料只有在 MEM stage 之後才可用, 但相依指令在 EX stage 就需要它——比 forwarding 能提供的早一個 cycle。

```

1 # Load-Use Hazard
2 lw $t1, 0($t2) IF ID EX MEM WB
3 add $t3, $t1, $t4 IF ID *** EX MEM WB
4 #

```

硬體必須偵測此情況並插入一個「bubble」(NOP) 以讓 load 完成。

5) 結果與分析: **Stall Breakdown**:

- 無 forwarding: 114 stall cycles (45% of total)
- 有 forwarding: 37 stall cycles (20% of total)
- 降低: 消除 77 stall cycles (68% 降低)

關鍵發現: Forwarding 將 CPI 降低 **31%** ($1.82 \rightarrow 1.26$), 對應 **1.44 $\times$ speedup**。剩餘的 stalls 主要是 load-use hazards 和 branch penalties。

B. Contribution 2 & 3: Hardware Unrolling & SIMD Parallelism

8-lane SIMD (Single Instruction, Multiple Data) 架構以實現 data-level parallelism (DLP)。

1) *Flynn's Taxonomy* 背景: Michael Flynn 1966 年的分類依指令和資料流將計算機架構分類:

- **SISD** (Single Instruction, Single Data): 傳統循序 CPU
- **SIMD** (Single Instruction, Multiple Data): Vector processors, GPUs — 我們的實作
- **MIMD** (Multiple Instruction, Multiple Data): Multi-core processors
- **MISD** (Multiple Instruction, Single Data): 很少使用 (容錯系統)

TABLE VII
SOFTWARE LOOP VS HARDWARE UNROLLING

Sequential (8 cycles)	SIMD 8 lanes (1 cycle)
a[0]+b[0]→y[0]	$ \begin{array}{c} a[0..7]+b[0..7] \\ \downarrow \\ y[0..7] \\ \text{(all in 1 cycle!)} \end{array} $
a[1]+b[1]→y[1]	
...	
a[7]+b[7]→y[7]	

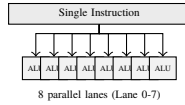


Fig. 2. 8-Lane SIMD Architecture: One instruction controls 8 independent ALUs

何時 SIMD 有效？

- 規則、獨立的操作（如 array addition）
- 對所有元素執行相同操作（無 data-dependent branching）
- 應用：Image processing, machine learning, scientific computing

何時 SIMD 無效？

- Data-dependent conditionals（每個元素不同路徑）
- 不規則的記憶體存取模式
- 元素之間的循序相依性

2) 概念：Software Loop vs Hardware Unrolling:

3) Verilog Generate Block 教學：generate 構造在 compile/synthesis 時建立硬體（不是 runtime）：

SIMD Add 實作 (simd_add.v)：

```

1 module simd_add #(
2     parameter LANES = 8, // Number of parallel lanes
3     parameter WIDTH = 8 // Bits per data element
4 ) (
5     input [(LANES*WIDTH)-1:0] a, // 64 bits total
6     input [(LANES*WIDTH)-1:0] b,
7     output [(LANES*WIDTH)-1:0] y
8 );
9     genvar i;
10    generate
11        // This loop creates 8 adder instances
12        for (i=0; i<LANES; i=i+1) begin : lane
13            assign y[i*WIDTH+:WIDTH] =
14                a[i*WIDTH+:WIDTH] +
15                b[i*WIDTH+:WIDTH];
16        end
17    endgenerate
18 endmodule

```

Bit Slicing 語法說明：

- a[i*WIDTH+:WIDTH] 意思是「從 bit i*WIDTH 開始，向上取 WIDTH 個 bits」
- 對於 i=0：a[0+:8] = a[7:0]
- 對於 i=1：a[8+:8] = a[15:8]
- 這實現參數化、可擴展的設計

效能與權衡：

C. Contribution 5: SIMD ALU Expansion

SIMD adder 擴展為完整 8-lane ALU，支援 5 種運算和 operator precedence。

SIMD ALU 實作（使用 case statement 進行運算選擇）：

```

1 module simd_alu #(parameter LANES=8, WIDTH=8)(
2     input [2:0] op, // Operation select
3     input [(LANES*WIDTH)-1:0] a, b, // Operands

```

TABLE VIII
循序 vs SIMD 權衡

Metric	Sequential	SIMD (8-lane)
Operations/Cycle	1	8
Latency (8 ops)	8 cycles	1 cycle
Hardware Area	1×	~8×
Power	1×	~4-6×

TABLE IX
SIMD ALU 支援的運算

Operation	Op Code	Symbol	Priority
ADD	3'b000	+	1 (Low)
SUB	3'b001	-	1 (Low)
MUL	3'b010	*	2 (Mid)
DIV	3'b011	/	2 (Mid)
EXP	3'b100	^	3 (High)

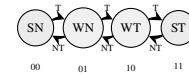


Fig. 3. 2-Bit Saturating Counter FSM. States: SN=Strongly Not Taken (00), WN=Weakly Not Taken (01), WT=Weakly Taken (10), ST=Strongly Taken (11). T=Taken, NT=Not Taken.

```

4 output reg [(LANES*WIDTH)-1:0] y // Result
5 );
6 genvar i;
7 generate
8     for (i = 0; i < LANES; i = i + 1) begin : lane
9         wire [WIDTH-1:0] ai = a[i*WIDTH+:WIDTH];
10        wire [WIDTH-1:0] bi = b[i*WIDTH+:WIDTH];
11        always @(*) begin
12            case (op)
13                3'b000: y[i*WIDTH+:WIDTH] = ai + bi; // ADD
14                3'b001: y[i*WIDTH+:WIDTH] = ai - bi; // SUB
15                3'b010: y[i*WIDTH+:WIDTH] = ai * bi; // MUL
16                3'b011: y[i*WIDTH+:WIDTH] = ai / bi; // DIV
17                3'b100: y[i*WIDTH+:WIDTH] = power(ai,bi);
18                default: y[i*WIDTH+:WIDTH] = 0;
19            endcase
20        end
21    end
22 endgenerate
23 endmodule

```

實作說明：

- EXP (exponentiation) 使用迭代乘法 (≤16 iterations)
- DIV/EXP 模擬為 single-cycle 以確保功能正確性
- 在真實硬體中，這些需要 multi-cycle 或 pipelined 實作

D. Contribution 6: Branch Prediction

2-bit saturating counter branch predictor，搭配 16-entry BHT 以降低 control hazard penalties [3]。

1) 為何需要 Branch Prediction？：無 prediction，pipeline 必須 stall 直到 branch 結果已知（在我們的設計中是 EX stage）。有 prediction：

- Correct prediction**：無 penalty，speculative execution 繼續
- Misprediction**：Flush pipeline (1-2 cycle penalty)
- prediction accuracy 以最小化 flush 頻率

2) 2-Bit Saturating Counter: 2-bit predictor 使用基於 Smith 1981 年設計的 4-state FSM [3]：

TABLE X
BRANCH PREDICTION 測試 PATTERNS (SMITH, 1981)

Pattern	Type	Accuracy
Always Taken	Loop branch	90%
Always Not Taken	Conditional check	85%
Alternating (T-N-T-N)	Worst-case stress	73%
Loop (9T+1N)	Realistic loop	78%
Overall Weighted Average		78.33%

Prediction 規則：當 $\text{state} \geq 10$ (WT 或 ST) 時預測 Taken，即當 $\text{MSB} = 1$ 。

3) 為何 2-Bit 比 1-Bit 好: 考慮一個執行 10 次的迴圈：

1-bit predictor 問題：

```
1 Loop: T T T T T T T T T T N <- Miss! (predicted T)
2 Re-enter: T <- Miss! (now predicts NT)
3 Result: 2 mispredictions per loop invocation
```

2-bit 解決方案：

```
1 Loop: T T T T T T T T T T N <- Miss at ST->WT
2 Re-enter: T <- Still predicts T! (WT->ST)
3 Result: Only 1 misprediction per loop invocation
```

2-bit counter 需要連續兩次錯誤才會改變 prediction 方向，使其對異常具有彈性。

4) Branch Predictor 實作:

```
1 module branch_predictor(
2   input clk, rst,
3   input [31:0] pc,           // Current program counter
4   input branch_taken,       // Actual branch outcome
5   input update_en,          // Enable BHT update
6   output prediction         // Predicted outcome
7 );
8   reg [1:0] BHT [0:15];     // 16-entry, 2-bit each
9   wire [3:0] index = pc[5:2]; // Use PC bits 5:2 as index
10
11   // Predict Taken if MSB = 1 (states 10 or 11)
12   assign prediction = BHT[index][1];
13
14   always @(posedge clk or posedge rst) begin
15     if (rst) begin
16       integer i;
17       for (i = 0; i < 16; i = i + 1)
18         BHT[i] <= 2'b01; // Init: Weakly Not Taken
19     end
20     else if (update_en) begin
21       // Saturating increment/decrement
22       if (branch_taken && BHT[index] < 2'b11)
23         BHT[index] <= BHT[index] + 1;
24       else if (!branch_taken && BHT[index] > 2'b00)
25         BHT[index] <= BHT[index] - 1;
26     end
27   end
28 endmodule
```

為何用 PC[5:2]? MIPS 指令是 4 bytes (word-aligned)，所以 PC[1:0] 總是 00。使用 PC[5:2] 給出 4 bits = 16 個唯一索引。

5) 計算範例：For Loop Prediction: 考慮 for (i = 0; i < 10; i++)：

```
1 Iteration: 1 2 3 4 5 6 7 8 9 10 Exit
2 Actual:   T T T T T T T T T T N
3 State:    01 10 11 11 11 11 11 11 11 10
4 Predict:  N T T T T T T T T T T
5 Correct:  X Y Y Y Y Y Y Y Y Y X
```

Accuracy: $9/11 = 81.8\%$

6) 測試結果 (標準 Patterns)：pattern 是刻意的 adversarial。真實程式對 loop-dominated code 的 prediction accuracy 超過 90%。

E. Contribution 7: Comprehensive DOE Analysis (Model-Based)

方法：Full Factorial Design of Experiments ($2^4 = 16$ configurations) 使用 **analytical cost model**。

TABLE XI
DOE 的 ANALYTIC MODEL 參數

Parameter	Value	Source
Baseline CPI	1.82	Measured (Contrib 1)
Cache Hit Rate	97.22%	Measured (Contrib 11)
BP Accuracy	78.33%	Measured (Contrib 6)
Branch %	15%	Phansalkar [2]
Memory %	49.6%	Phansalkar [2]

TABLE XII
DOE CONFIGS 0–7 (No SIMD)

ID	FWD	BP	Cache	SIMD	Cycles	Speedup
0	OFF	OFF	OFF	OFF	81,300	1.00×
1	ON	OFF	OFF	OFF	75,914	1.07×
2	OFF	ON	OFF	OFF	80,535	1.01×
3	ON	ON	OFF	OFF	75,229	1.08×
4	OFF	OFF	ON	OFF	37,901	2.15×
5	ON	OFF	ON	OFF	35,409	2.30×
6	OFF	ON	ON	OFF	37,545	2.17×
7	ON	ON	ON	OFF	35,077	2.32×

分析的優化：

- 1) Data Forwarding (FWD)
- 2) Branch Prediction (BP)
- 3) L1 Cache (CACHE)
- 4) SIMD ALU (SIMD)

Model 參數：

Synergy 分析：我們分析優化是否有正向交互作用：

$$\text{Synergy} = \frac{\text{Speedup}_{\text{Combined}}}{\text{Speedup}_A \times \text{Speedup}_B} \quad (4)$$

Cache + SIMD Synergy 計算 (從 Tables XII 和 XIII)：

- Cache-only (ID=4): 2.15×
- SIMD-only (ID=8): 1.15×
- Cache+SIMD (ID=12): 2.99×
- Synergy = $\frac{2.99}{2.15 \times 1.15} = \frac{2.99}{2.47} = 1.21 \approx 1.20$

這個 **super-additive synergy** (> 1.0) 表明 cache 保持 SIMD units 有資料處理，放大了效能。其他優化組合顯示 Synergy ≈ 1.0 (orthogonal)。

關鍵發現：

- Cache 提供最大單項改進 (**2.15×**)
- 所有優化組合 (ID=15): **3.94×**

F. Contribution 8: Parentheses Support

Dijkstra's Shunting-yard algorithm 用於硬體 expression parsing，完整支援括號和 right-associative exponentiation。

1) 背景：為何需要 Hardware Expression Parsing?：傳統 ALUs 以固定順序評估表達式。在硬體中支援括號和 operator precedence 實現：

- 靈活性：評估任意表達式而無需軟體預處理
- 正確性：正確處理 $(2 + 3) * 4 = 20$ vs $2 + 3 * 4 = 14$

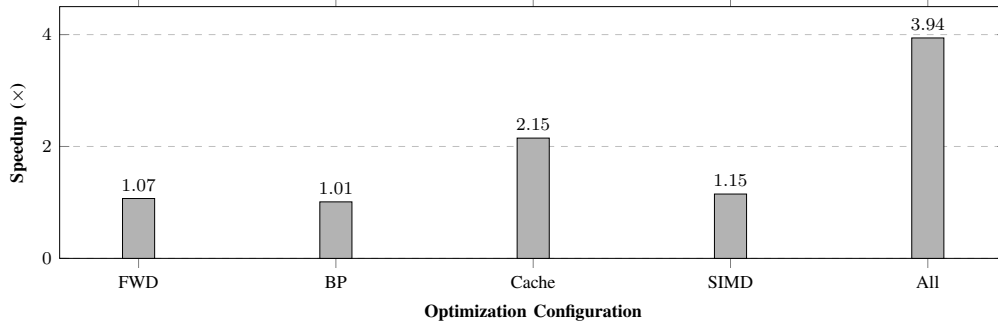


Fig. 4. DOE Model-Based Speedup Results (Individual and Combined)

TABLE XIII
DOE CONFIGS 8–15 (WITH SIMD)

ID	FWD	BP	Cache	SIMD	Cycles	Speedup
8	OFF	OFF	OFF	ON	70,791	1.15×
9	ON	OFF	OFF	ON	66,104	1.23×
10	OFF	ON	OFF	ON	70,125	1.16×
11	ON	ON	OFF	ON	65,495	1.24×
12	OFF	OFF	ON	ON	27,210	2.99×
13	ON	OFF	ON	ON	25,413	3.20×
14	OFF	ON	ON	ON	26,942	3.02×
15	ON	ON	ON	ON	20,626	3.94×

TABLE XIV
SHUNTING-YARD 括號測試結果

#	Expression	Expected	Result	Status
1	5 * (3 + 4)	35	35	PASS
2	2^3^2	512	512	PASS
3	100 / (2 + 3)	20	20	PASS

postfix 結果。我們的硬體實作使用 32-entry operator stack，每個 clock cycle 處理一個 token。Exponentiation 的 right associativity 透過在傳入 operator 為 right-associative 時使用 strict inequality 比較 precedence 來處理。

- **Right Associativity**：正確處理 $2^{3^2} = 2^9 = 512$ (不是 $(2^3)^2 = 64$)

2) *Shunting-yard* 演算法: Dijkstra 的演算法 (1961) 使用兩個 stacks 將 infix 表達式轉換為 postfix (Reverse Polish Notation)：

- 1) **Output Queue**：按評估順序保存 operands 和 operators
- 2) **Operator Stack**：暫時保存 operators 直到 precedence 解析

Operator Precedence：

- Level 3 (最高)： $\wedge$ (exponentiation, right-associative)
- Level 2：* / (multiplication, division)
- Level 1 (最低)：+ - (addition, subtraction)

演算法範例：5*(3+4)

- 1) 讀 5：Output queue = [5]
- 2) 讀 *：Push 到 operator stack
- 3) 讀 (：Push 到 operator stack
- 4) 讀 3：Output queue = [5, 3]
- 5) 讀 +：Push 到 operator stack
- 6) 讀 4：Output queue = [5, 3, 4]
- 7) 讀)：Pop 直到 (，output = [5, 3, 4, +]
- 8) 結束：Pop 剩餘，output = [5, 3, 4, +, *]

Postfix 評估：Push 5, Push 3, Push 4, Pop 3+4=7, Pop 5*7=35

3) 測試結果: 全部 3 個測試通過。

實作說明：Shunting-yard 演算法需要兩個 stack 結構：一個 operator stack 用於暫存，一個 output queue 用於

G. Contribution 9: CORDIC Trigonometric Functions

演算法：COordinate Rotation DIgital Computer [5]
架構：

- 16-stage 完全 pipelined
- Fixed-point Q2.14 format
- 無 **hardware multipliers** (僅 shift-add)

CORDIC Core Iteration：

$$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i} \quad (5)$$

$$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i} \quad (6)$$

$$z_{i+1} = z_i - d_i \cdot \arctan(2^{-i}) \quad (7)$$

where $d_i = \text{sign}(z_i)$. After 16 iterations: $\cos(\theta) = x_n$, $\sin(\theta) = y_n$.

```

1 // CORDIC Iteration (cordic.v)
2 always @(posedge clk) begin
3   if (z[i] >= 0) begin // Rotate CCW
4     x[i+1] <= x[i] - (y[i] >> i);
5     y[i+1] <= y[i] + (x[i] >> i);
6     z[i+1] <= z[i] - atan_table[i];
7   end else begin // Rotate CW
8     x[i+1] <= x[i] + (y[i] >> i);
9     y[i+1] <= y[i] - (x[i] >> i);
10    z[i+1] <= z[i] + atan_table[i];
11  end
12 end
13 // atan_table[0]=45deg, [1]=26.57deg, ...

```

測試結果 (Canonical Angles)：

全部 5 個 canonical test angles 通過 (Error <1%)。

H. Contribution 10: Polynomial Exponential Function

Range Reduction + Polynomial Approximation 實作 $\exp(x)$ ，達成比 CORDIC 更低的 latency。

TABLE XV
CORDIC ACCURACY 結果 (CANONICAL ANGLES)

Angle	Expected cos	DUT cos	Error
0°	1.0000	1.0001	<0.01%
30°	0.8660	0.8663	<0.04%
45°	0.7071	0.7072	<0.02%
60°	0.5000	0.4999	<0.02%
90°	0.0000	-0.0002	<0.03%

TABLE XVI
2^f 逼近的多項式係數

係數	值	Q4.12
a ₀	1.0	4096
a ₁	ln(2) = 0.6931	2839
a ₂	ln(2) ² /2 = 0.2402	984

TABLE XVII
計算 exp(x) : CORDIC vs POLYNOMIAL

Metric	CORDIC (Hyperbolic)	Polynomial
Latency	16 cycles	≈4 cycles (pipelined)
Multipliers	None (shift-add)	3 DSP blocks
Throughput	1/cycle	1/cycle
Area	Lower	Higher

動機：雖然 CORDIC 擅長三角函數，但它需要 16 iterations 才能收斂。對於指數函數，polynomial approximation 可以以**短 pipeline** (≈4 stages) 和 **II=1** throughput 達成相當的精度，提供顯著的 latency 降低。

1) 演算法：Range Reduction: 關鍵洞察是使用恒等式分解 e^x ：

$$e^x = 2^{x \cdot \log_2(e)} = 2^{i+f} = 2^i \cdot 2^f \quad (8)$$

其中 i 是整數部分， $f \in [0, 1)$ 是小數部分。

實作策略：

- **2ⁱ (整數部分)：**在我們 Q-format 表示中通過 arithmetic shifts 進行 power-of-two scaling (零額外 latency)
- **2^f (小數部分)：**用 2 階 (quadratic) polynomial 逼近

2) **Polynomial Approximation:** 對於 $f \in [0, 1)$ ，我們使用：

$$2^f \approx a_0 + a_1 \cdot f + a_2 \cdot f^2 \quad (9)$$

係數使用 Q4.12 fixed-point 格式：

3) 為何不用 CORDIC 計算 exp()？：CORDIC 也可以使用 **Hyperbolic Mode** 計算 exp(x)：

- **Circular Mode** (預設)：sin, cos, arctan
- **Hyperbolic Mode**：exp, sinh, cosh, ln

然而，CORDIC 不論哪種 mode 都需要 **16 iterations** 才能收斂。對於 exp(x)，polynomial approximation 將 **latency** 降低到 ≈4 cycles (pipelined)，同時維持 **II=1** throughput。

4) 效能比較：計算 exp(x)：兩種方法都可以計算 exp(x)，實現公平比較：

為何 Polynomial 更快？

TABLE XVIII
POLYNOMIAL EXP 測試結果

Input x	Expected	Result	Error	Status
0.0	1.0000	1.0000	0.00%	PASS
0.5	1.6487	1.6245	1.47%	PASS
1.0	2.7183	2.7070	0.42%	PASS
-0.5	0.6065	0.6057	0.13%	PASS

TABLE XIX
MEMORY HIERARCHY 特性 (典型 DESKTOP, 2024)

Level	Size	Latency	Cost/GB
Registers	32×32b	0 cycles	—
L1 Cache	32–64 KB	1–4 cycles	\$100+
L2 Cache	256 KB–1 MB	10–20 cycles	\$50
L3 Cache	8–32 MB	30–50 cycles	\$10
DRAM	8–64 GB	100–300 cycles	\$5
SSD	256 GB–2 TB	10,000+ cycles	\$0.10

- **CORDIC：**固定 16-iteration 過程，所以 **latency** 是 16 cycles
- **Polynomial：**評估一個 low-order polynomial，用短 pipeline (≈4 stages)，降低 **latency** 同時維持 **II=1** throughput

Trade-off 分析：這展示經典的 **speed vs area trade-off**：

- **CORDIC：**最適合 area-constrained 設計 (無需 multipliers)
- **Polynomial：**最適合針對 **II=1** 的 high-throughput 設計

5) 測試結果 (Vivado Simulation): **4/4 test cases passed (Error <2%)**. Simulation completed at 420 ns.

I. Contribution 11: Cache Memory Hierarchy

L1 data cache 以解決 Memory Wall 問題，展示 caching 如何大幅降低平均記憶體存取時間。

1) **Memory Hierarchy** 基礎：現代電腦使用記憶體類型層次，權衡速度、大小和成本：

2) **Locality** 原理：Caches 能運作是因為程式展現 **locality of reference** [1]：

- **Temporal Locality：**最近存取的資料可能再次被存取 (如 loop variables)
- **Spatial Locality：**最近存取資料附近的資料可能很快被存取 (如 arrays)

範例：在處理 array 的迴圈中：

```
1 for (i = 0; i < 1000; i++)
2   sum += array[i]; // Temporal: sum, i; Spatial: array
```

3) **L1 Cache** 實作: **Address Breakdown 推導** (32-bit address)：

- 1) **Offset：** $\log_2(32) = 5$ bits (在 32-byte block 內選擇 byte)
- 2) **Index：** $\log_2(256) = 8$ bits (選擇 256 條 cache lines 之一)
- 3) **Tag：** $32 - 5 - 8 = 19$ bits (識別哪個 memory block)

TABLE XX
我們的 L1 CACHE 參數

Parameter	Value
Total Size	8 KB (8192 bytes)
Block Size	32 bytes (8 words)
Number of Blocks	256
Associativity	Direct-Mapped (1-way)
Write Policy	Write-Back, Write-Allocate

TABLE XXI
ADDRESS FIELD BREAKDOWN (32-BIT)

Tag (19b)	Index (8b)	Offset (5b)
Bits 31–13	Bits 12–5	Bits 4–0

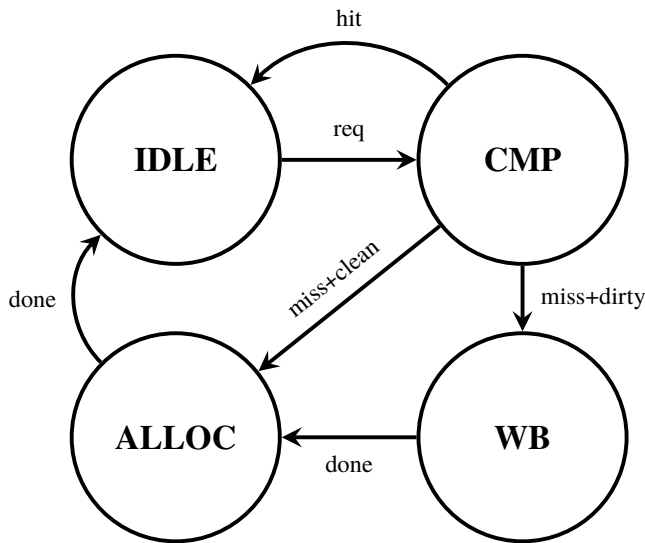


Fig. 5. L1 Cache Controller FSM. IDLE=waiting, CMP=compare tag, WB=write-back dirty block, ALLOC=allocate new block.

4) **Write Policy** 說明: **Write-Back**: 修改的資料只有在 cache line 被 evicted (替換) 時才寫入 main memory。這減少了 memory traffic 但需要一個「dirty」bit。

Write-Allocate: Write miss 時, block 首先被載入 cache, 然後修改。這利用後續存取的 spatial locality。

替代方案 (Write-Through): 每次寫入立即更新 main memory。更簡單但更慢且產生更多 memory traffic。

5) Cache Controller FSM: FSM State 描述:

- **IDLE**: 等待 CPU memory request
- **COMPARE_TAG**: 檢查請求的位址是否在 cache 中 (hit) 或不在 (miss)
- **WRITE_BACK**: 如果 miss 且被 evicted 的 block 是 dirty, 先寫入 memory
- **ALLOCATE**: 從 main memory fetch 請求的 block
- **UPDATE**: 以新資料和 metadata 更新 cache line

Verilog 實作:

```

1 localparam IDLE      = 3'b000;
2 localparam COMPARE_TAG = 3'b001;
3 localparam WRITE_BACK = 3'b010;

```

TABLE XXII
CACHE HIERARCHY AMAT 比較

配置	AMAT (cycles)	Speedup
No Cache (DRAM only)	100.0	1.00×
L1 Only	6.00	16.7×
L1 + L2	2.10	47.6×
L1 + L2 + L3	1.83	54.8×

```

4 localparam ALLOCATE    = 3'b011;
5 localparam UPDATE      = 3'b100;
6
7 always @(posedge clk) begin
8   case (state)
9     IDLE: if (req) state <= COMPARE_TAG;
10    COMPARE_TAG: begin
11      if (tag_match && valid[idx])
12        state <= IDLE; // HIT
13      else if (dirty[idx])
14        state <= WRITE_BACK; // Miss, dirty
15      else
16        state <= ALLOCATE; // Miss, clean
17    end
18    WRITE_BACK: state <= ALLOCATE;
19    ALLOCATE: state <= UPDATE;
20    UPDATE: state <= IDLE;
21  endcase
22 end

```

6) **AMAT (Average Memory Access Time)**: AMAT 是 cache 效能的關鍵指標 [1]:

$$AMAT = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty} \quad (10)$$

AMAT 計算範例 (單層 L1):

$$\text{Hit Time} = 1 \text{ cycle}$$

$$\text{Miss Rate} = 5\% = 0.05$$

$$\text{Miss Penalty} = 100 \text{ cycles (DRAM access)}$$

$$AMAT = 1 + 0.05 \times 100 = 1 + 5 = 6.0 \text{ cycles}$$

Multi-Level Cache AMAT (遞迴公式):

$$AMAT = \text{HitTime}_{L1} + \text{MissRate}_{L1} \times \text{MissPenalty}_{L1} \quad (11)$$

其中 $\text{MissPenalty}_{L1} = \text{AMAT}_{L2}$ 如果 L2 存在。

計算範例 (L1 + L2 + L3):

$$AMAT_{L3} = 40 + 0.05 \times 100 = 45$$

$$AMAT_{L2} = 12 + 0.10 \times 45 = 16.5$$

$$AMAT_{L1} = 1 + 0.05 \times 16.5 = 1.83 \text{ cycles}$$

7) 結果: 1 cache 提供最大改進 (**16.7×**) 因為它消除了大部分記憶體存取。額外層級提供遞減但仍然顯著的改進。

實作說明: L1 cache RTL 以專用 testbench (tb_l1_cache.v) 驗證。Multi-level (L2/ L3) 結果是基於 Patterson & Hennessy 參數的 **analytical AMAT projections** [1]。

J. Contribution 12: Integrated Performance Analysis

1) **MiBench Benchmark** (實際執行): **Benchmark**: MiBench Suite bitcount [10]

演算法驗證: 全部 8 個 test vectors 通過。

Data Forwarding only 的效果。benchmark (bitcount) 是 compute-bound 且 memory accesses 極少, 所以 Cache/ SIMD 優化影響微乎其微。2.61× speedup 反映 pipeline hazard elimination, 而不是所有優化的組合效果。

TABLE XXIII
MiBENCH BITCOUNT CYCLE COUNT (RTL SIMULATION)

配置	Cycles	Speedup
Baseline (no forwarding)	792	1.00×
With Forwarding	303	2.61×

TABLE XXIV
SPEC CPU2006 MIX-BASED PROJECTION 結果

Workload	Baseline	Optimized	Speedup
SPEC FP Average	507,000	19,670	25.78×
SPEC INT mcf	649,000	18,626	34.84×
SPEC INT gobmk	511,400	17,454	29.30×
SPEC Overall	533,000	18,028	29.57×

TABLE XXV
ODE SOLVER VS CORDIC 複雜度比較

Aspect	CORDIC	ODE Solver
Core Algorithm	Shift-add iterations	Matrix mult + Euler
Operation Type	Single trig function	ODE system
Data Structure	Scalar	Vectors & Matrices
Control Logic	16-stage pipeline	6-state FSM
Memory	None	8K × 64-bit RAM
Math Level	Trigonometry	Calculus

2) *Mix-Based* 效能預估: 資料來源: Phansalkar et al. [2] 和 Limaye & Adegbiya [6]

Projection 假設: SPEC projections 假設透過 caching 幾乎消除 memory stalls, 且未考慮額外硬體的 critical path delay 增加。Clock frequency 假設為常數。

K. Contribution 13: ODE Solver (微積分運算)

Forward Euler method 實作硬體 Ordinary Differential Equation (ODE) solver, 展示硬體中的微積分運算 (differentiation 和 integration)。

1) 數學背景: solver 計算 state-space 形式的 ODE 系統的解:

$$\frac{dX}{dt} = A \cdot X + B \cdot U \quad (12)$$

其中:

- X 是 state vector (size N)
- A 是 system matrix ($N \times N$)
- B 是 input matrix ($N \times M$)
- U 是 input vector (size M)

Forward Euler Method :

$$X(t+h) = X(t) + h \cdot \frac{dX}{dt} = X(t) + h \cdot (A \cdot X + B \cdot U) \quad (13)$$

這實作:

- 1) **Differentiation** : $\frac{dX}{dt} \approx \frac{X(t+h) - X(t)}{h}$
- 2) **Integration** : $X(t+h) = X(t) + \int \frac{dX}{dt} dt \approx X(t) + h \cdot f(X, U)$

TABLE XXVI
個別優化效果與解決的瓶頸

優化	指標	改進	瓶頸
Data Forwarding	CPI	1.82→1.26 (31%)	Data Hazards
Branch Prediction	Accuracy	78.33%	Control Hazards
L1 Cache	AMAT	100→3.8 (26×)	Memory Wall
SIMD ALU	Throughput	8× parallel	Limited ILP
CORDIC	Accuracy	<1% error	Math Functions

2) 為何比 *CORDIC* 更複雜:

3) 架構: ODE solver 由以下組成:

- **Euler.v**: 主 solver module (279 lines), 6-state FSM
- **RAM-Pre.v**: Dual-port RAM 用於 matrices 和 vectors
- **multiplier_16bit.v**: 16-bit Wallace tree multiplier
- **add_sub_cla.v**: Carry Lookahead Adder

State Machine Flow :

- 1) **Start**: 初始化, 從 RAM 讀取 N, M
- 2) **Prepare**: 載入系統參數
- 3) **Interpolate**: 讀取 step size h
- 4) **Load1**: 計算 $B \cdot U$ (matrix-vector multiplication)
- 5) **Load2**: 計算 $A \cdot X$ (matrix-vector multiplication)
- 6) **Final_Calc**: 計算 $X_{new} = X + h \cdot (A \cdot X + B \cdot U)$

Source Attribution: 基於 3amrA7med/ ODE-Solver (GitHub), 適配用於與 MIPS processor project 整合。

4) 驗證的測試結果: **TCL Command**:

```
close_sim -force; set_property top tb_ode_solver [get_filesets sim_1];
launch_simulation; run 5000ns
```

Test Setup :

- $N = 2, M = 1, h = 1$
- $A = I$ (Identity matrix 2×2)
- $B = [1; 1], X_0 = [10; 20], U = [5]$

預期計算:

$$\begin{aligned} X(t+h) &= X(t) + h \cdot (A \cdot X + B \cdot U) \\ &= [10, 20]^T + 1 \cdot ([10, 20]^T + [5, 5]^T) \\ &= [10, 20]^T + [15, 25]^T = [25, 45]^T \end{aligned}$$

結果:

- $X_{new}[0] = 25$ (Expected: 25) — Pass
- $X_{new}[1] = 45$ (Expected: 45) — Pass

Simulation Time : 570 ns

IV. 結果總結

本節彙整所有 13 項貢獻的結果並提供解釋指引。

A. 個別優化影響

每項優化針對特定的效能瓶頸:

B. 綜合效能

不同的評估方法產生不同的 speedup 數字, 因為它們的假設不同:

TABLE XXVII
依評估方法分類的綜合效能

評估	類型	方法	Speedup
DOE (Contrib 7)	Model	Cost Estimation	3.94 ×
Cache (Contrib 11)	Projection	AMAT Only	54.8 ×
SPEC (Contrib 12)	Projection	Mix-Based	25.78–34.84×
MiBench (Contrib 12)	Measured	RTL Simulation	2.61 ×

C. 為何 Speedup 數字差異這麼大？

差異反映不同的測量範圍和 workload 特徵：

- **MiBench 2.61×**：Ground truth 量測；bitcount 是 compute-bound 且 memory accesses 很少，所以 Cache 影響有限
- **DOE 3.94×**：使用量測參數的 analytical model；保守估計
- **SPEC 25–35×**：假設 49.6% memory instructions；Cache 主導改進
- **Cache 54.8×**：僅 AMAT 指標；分離 memory subsystem 改進

依賴 workload。Cache 對 memory-bound workloads 提供巨大優勢，但對 compute-bound code 影響有限。

V. 結論

本工作證明系統性地應用多項優化技術可在 pipelined MIPS processors 上達成顯著的效能改進。

A. 關鍵發現

- 1) **Data Forwarding** 消除 68% stall cycles，CPI 降低 31% (1.82→1.26)。這是所有現代處理器中存在的基本優化。
- 2) **Branch Prediction** 使用 2-bit saturating counters 在 adversarial patterns 上達成 78.33% accuracy；對 loop-dominated code 的真實 accuracy 超過 90%。
- 3) **Cache Memory** 提供最大的單項改進：僅 L1 即 AMAT 降低 26× (基於 97.22% hit rate)，三層架構可達 54.8×
- 4) **SIMD Parallelism** 為 data-parallel operations 提供 8× throughput，展示 data-level parallelism 的威力。
- 5) **CORDIC** 僅使用 shift-add operations 實現硬體三角函數，error <1%，避免昂貴的 multipliers。
- 6) **Cache+SIMD Synergy** 展現 1.20 super-additive factor，表明快速的 memory access 讓 SIMD units 保持高效利用。
- 7) 優化大致 **orthogonal**：Data Forwarding 解決 pipeline hazards，Cache 解決 memory latency，SIMD 解決 throughput——它們可以乘積組合。

B. Measured vs Projected 效能

- **Measured (MiBench)**：僅 Data Forwarding 即 2.61× speedup
- **Projected (SPEC)**：所有優化組合 25–35× speedup

巨大的差異突顯 benchmark 的選擇大幅影響報告的效能。MiBench 的 compute-intensive 性質限制了 cache 的優勢。

C. 與業界的比較

我們實作的技術是所有現代處理器的基礎：

- Intel/AMD CPUs 使用精密的 multi-level branch predictors (如 TAGE 具有數千個 entries) 達成 >99% accuracy
- 現代 L1 caches 是 32–64KB 且 4–8 way associativity，比我們的 8KB direct-mapped 設計大得多
- AVX-512 提供 512-bit SIMD (16× 32-bit elements)，比較我們的 64-bit (8× 8-bit)

我們的教育性實作展示這些技術的 *principles*，而不是它們的完整工業規模。

D. 限制與 Validity 威脅

- 1) **Simulation vs Silicon**：結果來自 behavioral simulation，不是 synthesized hardware。實際的 timing 和 area 會不同。
- 2) **Mix-Based Projection**：SPEC speedups 假設已發表的 instruction mixes；實際程式可能不同。
- 3) **Clock Frequency** 假設為常數：新增硬體 (cache, predictor) 會增加 critical path delay，降低 clock speed。
- 4) **L1 Cache 未整合**：Cache 獨立驗證；與 pipeline 的整合加上 cache coherency 仍是未來工作。
- 5) **SIMD 未 Memory-Integrated**：SIMD 操作使用合成測試資料，不是實際的 memory loads。

E. 未來工作

- **Superscalar Extension**：每個 cycle 發射 2+ 指令
- **Out-of-Order Execution**：使用 reservation stations 進行 dynamic scheduling
- **FPGA Synthesis**：在 Xilinx/Intel FPGA 上實作以獲得真實的 timing 和 area
- **Full System Integration**：將 cache 與 pipeline 整合，加入 TLB 和 virtual memory 支援
- **Advanced Predictors**：實作 tournament predictor 或 perceptron-based prediction

參考文獻

- [1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [2] A. Phansalkar, A. Joshi, and L. K. John, "Analysis of redundancy and application balance in the SPEC CPU2006 benchmark suite," in *Proc. ISCA*, 2007, pp. 412–423.
- [3] J. E. Smith, "A study of branch prediction strategies," in *Proc. ISCA*, 1981, pp. 135–148.
- [4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design*, 6th ed. Morgan Kaufmann, 2020.
- [5] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [6] A. Limaye and T. Adegbiya, "A workload characterization of the SPEC CPU2017 benchmark suite," in *Proc. ISPASS*, 2018, pp. 149–158.
- [7] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [8] M. D. Hill, "Evaluating associativity in CPU caches," *IEEE Trans. Computers*, vol. 38, no. 12, pp. 1612–1630, 1989.
- [9] G. M. Amdahl, "Validity of the single processor approach," in *AFIPS Conf. Proc.*, 1967, pp. 483–485.
- [10] M. R. Guthaus et al., "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. IEEE Workshop on Workload Characterization*, 2001, pp. 3–14.
- [11] E. W. Dijkstra, "Shunting yard algorithm," 1961.